

Matplotlib & Seaborn Fundamentals



Suman

26 Feb, 2026 At 8:00 PM

Notes

Module-1

Recommended

Resource



Associated Lectures (1)



Description



Discussions

Lecture Notes: Matplotlib & Seaborn Fundamentals

PPT File 1: [Click Here](#)

PPT File 2: [Click Here](#)

Colab File: [Click Here](#)

Advanced Wrangling to Visual Foundations

Program: Vishlesan i-Hub IIT Patna x Masai School -- AIM (AI & Machine Learning) **Session ID:** 4.5 I
Week: 4/5 Transition **Prerequisites:** Pandas Fundamentals

Mental Map

Session 4.5: Advanced Wrangling to Visual Foundations

```
|  
+-- A. COMBINING DATAFRAMES: CONCAT  
|   +-- pd.concat() - "dumb gluing" (stacking)  
|   +-- Vertical stacking (axis=0) vs Horizontal (axis=1)  
|   +-- ignore_index=True - prevent duplicate indices  
|  
+-- B. COMBINING DATAFRAMES: MERGE/JOIN  
|   +-- pd.merge() - Smart lookup (VLOOKUP on steroids)  
|   +-- on=, left_on=, right_on= for key columns  
|   +-- The 4 Join Types: INNER, LEFT, RIGHT, OUTER  
|   +-- Many-to-many danger: Cartesian product / row explosion  
|  
+-- C. GROUPBY & AGGREGATION  
|   +-- Split-Apply-Combine pattern
```

```
|   +-- .groupby('col')['val']  
|   +-- .agg() – multiple aggregations, named aggregations  
|   +-- .pivot_table() – cross-tabulation summaries  
|  
+-- D. MATPLOTLIB ARCHITECTURE  
    +-- Figure / Axes / Artist Hierarchy  
    +-- pyplot vs Object-Oriented Interface  
    +-- Why the Object-Oriented model is mandatory for professionals
```

The Big Picture

Data analysis rarely happens in a single, perfectly formatted table. In the real world, your data is scattered across multiple databases, logs, and files. Before you can visualize insights, you must bring the pieces together and summarize them.

This session bridges the gap between raw data manipulation and visual storytelling. First, you will learn to stitch disparate datasets together using `pd.concat` and `pd.merge`. Next, you will learn to condense millions of rows into actionable summaries using `groupby`, `.agg()`, and `pivot_table`.

Finally, we transition from the "Data Wrangler" to the "Data Communicator" by introducing the mental model that powers Python's foundational visualization library: the Matplotlib Figure and Axes architecture. Understanding this underlying structure is the key to creating publication-quality charts.

Main Content

Topic A: Combining DataFrames — Concat

Core Message: `pd.concat()` is for stacking DataFrames together when they share the same structure but contain different batches of data. It does not match data based on a key; it is purely structural "gluing."

Vertical vs. Horizontal Stacking

- **Vertical Stacking (`axis=0` , default):** Adding more *rows*. Example: Appending February's sales data to January's sales data.
- **Horizontal Stacking (`axis=1`):** Adding more *columns*. Example: Gluing a 100-row table of customer names to a 100-row table of customer addresses (assuming the order matches perfectly).

Code: The `ignore_index` Gotcha

When stacking vertically, Pandas keeps the original row index numbers by default, leading to duplicate indices (e.g., 0, 1, 2, 0, 1, 2).

```
import pandas as pd  
  
# ALWAYS use ignore_index=True for vertical stacking
```

```
combined_df = pd.concat([jan_sales, feb_sales], axis=0, ignore_index=True)
```

Topic B: Combining DataFrames — Merge

Core Message: `pd.merge()` is a relational join. It reaches into another table and pulls information based on a matching key column (like a `VLOOKUP` in Excel, but highly scalable).

The Four Join Types

The `how` parameter dictates which rows survive the merge if there isn't a perfect match.

INNER JOIN (`how='inner'` - Default): Only rows with matching keys in BOTH tables survive. (e.g., "Show me only the orders where we have a valid customer profile").

LEFT JOIN (`how='left'` - Most Common): Keeps ALL rows from the left table, and brings in data from the right table where it matches. Unmatched left rows get `NaN` for the new columns. (e.g., "Keep all my orders, and attach customer data if we have it").

RIGHT JOIN (`how='right'`): Keeps ALL rows from the right table, bringing in matching data from the left.

OUTER JOIN (`how='outer'`): Keeps EVERYTHING. All rows from both tables are kept; any missing matches are filled with `NaN`.

Code: Standard Left Merge

```
# Enriching an orders table with customer details
enriched_orders = pd.merge(
    left=orders_df,
    right=customers_df,
    on='customer_id',
    how='left'
)
```

Warning: Always check for row explosions! If the `customer_id` is not unique in the `customers_df`, `pd.merge` will create a Cartesian product, multiplying your rows and silently inflating your data.

Topic C: GroupBy, Aggregation & Pivot Tables

Core Message: Summarization requires the Split-Apply-Combine methodology. You split the data into categories, apply a mathematical function, and combine it back into a summary table.

1. GroupBy and .agg()

The `.agg()` method allows you to perform multiple different mathematical operations on multiple columns simultaneously.

Code: Named Aggregations (Industry Standard)

```
# What is the total revenue and average quantity sold per city?
city_summary = sales_df.groupby('city').agg(
    total_revenue=('sales_amount', 'sum'),
    avg_quantity=('quantity', 'mean'),
    transaction_count=('order_id', 'count')
).reset_index() # ALWAYS reset_index to return a clean DataFrame
```

2. Pivot Tables

While `.groupby()` creates a flat summary (one category per row), `pd.pivot_table()` creates a cross-tabulation grid, making it easier to compare sub-categories.

Code: Cross-Tabulation

```
# rows = city, columns = product_category, values = sales_amount
sales_matrix = pd.pivot_table(
    data=sales_df,
    values='sales_amount',
    index='city',
    columns='product_category',
    aggfunc='sum',
    fill_value=0 # Replace missing combinations with 0
)
```

Topic D: Matplotlib Architecture

Core Message: To communicate the aggregations you just built, you must visualize them. Matplotlib is built on a strict hierarchy. Do not rely on quick scripts; understand the Object-Oriented architecture to build professional layouts.

The Three-Layer Hierarchy

Figure (`fig`): The master canvas. It is the entire window or the final image file you save.

Axes (`ax`): The individual plot area. This is where the data lives. A Figure can have one Axes (a single chart) or many Axes (a dashboard subplot). *Crucially: "Axes" is the bounding box, not the X/Y number lines.*

Artist: Everything you can see on the Axes (lines, bars, text, labels, ticks).

The Professional Standard: Object-Oriented API

Never use `plt.plot()` for final deliverables. It relies on implicit state and breaks down when building complex dashboards. Always create your Figure and Axes explicitly.

Code: The OO Mental Model

```
import matplotlib.pyplot as plt
```

```
# 1. Create the Figure (Canvas) and Axes (Plot area)
fig, ax = plt.subplots(figsize=(8, 5))

# 2. Add Artists to the Axes
ax.bar(city_summary['city'], city_summary['total_revenue'], color='#ED1C24')

# 3. Customize the Axes
ax.set_title('Total Revenue by City', fontweight='bold')
ax.set_ylabel('Revenue (INR)')

# 4. Render the Figure
plt.show()
```

Key Frameworks

Framework 1: Combine vs. Merge

Scenario	Tool to Use	Mental Model
I have January data and February data.	<code>pd.concat(axis=0)</code>	Stacking cafeteria trays vertically.
I have a table of Orders and a table of Customer Info.	<code>pd.merge()</code>	<code>VL00KUP</code> on a shared ID key.

Framework 2: Summarization Selection

Goal	Tool to Use	Output Structure
Summarize a single metric for a single category.	<code>.groupby()</code> <code>['col'].sum()</code>	Flat Series.
Calculate multiple different metrics for a category.	<code>.groupby().agg()</code>	Flat DataFrame (after <code>reset_index</code>).
Compare sub-categories in a grid format.	<code>pd.pivot_table()</code>	2D Matrix (Rows & Columns).

Common Errors Quick Reference

#	Error	What It Means	How to Fix It
1	Duplicate indices after <code>pd.concat</code>	Stacked rows kept their original row numbers.	Add <code>ignore_index=True</code> .
2	Rows mysteriously multiplied after <code>pd.merge</code>	The <code>on</code> key has duplicates in both tables	Deduplicate your lookup table before merging.

#	Error	What It Means	How to Fix It
		(Many-to-Many).	
3	GroupBy result has the category as the index, not a column	Missing the final cleanup step.	Chain <code>.reset_index()</code> at the very end of your <code>.agg()</code> .
4	<code>AttributeError: 'Figure' object has no attribute 'plot'</code>	Tried to plot data onto the canvas directly, instead of the plot area.	Call <code>.plot()</code> or <code>.bar()</code> on the <code>ax</code> object, not the <code>fig</code> object.

Key Takeaways

Stitching Data: Use `pd.concat` for structural stacking (adding batches of data). Use `pd.merge` for relational lookups based on shared keys.

Default to Left Joins: When merging, `how='left'` is the industry standard to ensure you do not drop records from your primary table of interest just because a lookup failed.

The Power of `.agg()`: The Split-Apply-Combine pattern is how raw data becomes business intelligence. Use named aggregations in `.agg()` to cleanly define your output column names in one step.

Figure vs. Axes: To plot your summarized data effectively, memorize the Matplotlib hierarchy. The Figure is the blank piece of paper; the Axes is the specific square where the ink is drawn. Always use the explicit Object-Oriented interface (`fig, ax = plt.subplots()`).